NEXORA

Smart Shortlisting Engine

Project Report

InternLoom AI Hackathon — Manipal Institute of Technology, Bengaluru

Prepared by

Suryansh Verma
(245805436: 3RD YEAR): MIT CSE CORE C
B.Tech Computer Science & Engineering,
MIT Bengaluru

September 2026

Built 100% offline — zero external API calls

Table of Contents

1. Executive Summary
2. Problem Statement
3. Objectives
4. System Architecture
5. Methodology
6. Explainability Layer
7. Bonus Feature — JD Bias & Narrow-Phrasing Review
8. Bonus Feature — Recruiter “Compare Candidates” Tool
9. User Interface Walkthrough
10. Compliance with Hackathon Requirements
11. Offline Execution & Privacy
12. Testing & Validation
13. Challenges Faced & Solutions
14. Future Enhancements
15. Conclusion

1. Executive Summary

Nexora is a Smart Shortlisting Engine built for the InternLoom AI Hackathon. Given a single job description and a batch of resumes, it produces a complete, ranked, and explainable shortlist of candidates — combining semantic understanding of meaning and context with explicit keyword and skill matching, exactly as the problem statement requires.

The system is designed around three non-negotiable principles that shaped every implementation decision: the ranking must genuinely use both semantic and keyword signals rather than a single black-box score; every decision the system makes — a rank, a missing skill, a discarded duplicate — must be explainable in plain language; and the entire pipeline must run locally, with zero calls to any external API or hosted LLM at any stage.

Beyond the core requirement, Nexora implements both optional bonus features described in the problem statement — a heuristic JD bias and narrow-phrasing scanner, and a recruiter-facing "Compare Candidates" natural-language Q&A tool — along with additional robustness work: typo-tolerant matching, date-format normalization, transparent handling of candidates who submit more than one resume file, and automatic role-applied-for detection per resume.

2. Problem Statement

2.1 Context

Campus placement platforms such as InternLoom rank an entire pool of applicants against a single job opening before a human recruiter ever looks at them. A strong candidate rarely describes themselves using the exact words in a job description — someone who has "built REST APIs with Express and MongoDB" is clearly relevant to a Node.js backend role even if the JD never uses the word "Express." At the same time, specific tools and technologies named in a JD still matter: a role that explicitly asks for certain skills should not be satisfied by only loosely related experience. Resumes also vary widely in structure, phrasing, and completeness, and any ranking produced needs to be justifiable, not just a bare number.

2.2 What Had to Be Built

Given one job description and a batch of 15–18 resumes, the system was required to:

1. Evaluate every resume against the JD using an approach that draws on both semantic search (meaning and context) and keyword search (explicit skills, tools, and terms) — both genuinely factoring into the final result.
2. Produce a ranked list of all candidates from best fit to worst fit, with a final score for each.
3. For the top 3 ranked candidates, generate a short explanation of why they ranked there — which skills matched and which required skills appear to be missing.

> **Important Constraint**
>
> *"Simply pasting a resume and JD into an LLM API and asking it to give this resume a score out of 100 does not meet the requirement and will not be scored under the core criteria. Your own system needs to genuinely perform the semantic and keyword matching as part of its logic." — Nexora satisfies this by running its own local embedding model and its own regex/dictionary keyword logic; no resume or JD text is ever sent to an external LLM or API.*

2.3 Bonus Features (Optional)

- Flag potential bias or overly narrow phrasing in the JD itself that could unfairly exclude qualified candidates.
- Build a simple UI or chat layer where a recruiter can ask, e.g., “Why is Candidate X ranked above Candidate Y?” and get a natural-language answer.
- Handle messy or inconsistent resume formatting gracefully (typos, varied date formats, inconsistent section headers, etc.).

2.4 Provided Dataset

The hackathon supplied one sample job description (Sample_JD.pdf — a Junior Full Stack Developer Intern role at a fictional company, TechNova Solutions) and 18 sample resumes in PDF format, deliberately spanning strong, partial, and weak matches so that a correct ranking system shows meaningful spread rather than near-identical scores.

2.5 Judging Rubric

Criterion	Weight
Effective use of both semantic and keyword-based matching	35%
Quality and sensibility of the overall ranking produced	20%
Accuracy and clarity of top-3 candidate explanations	20%
Working end-to-end demo	15%
Bonus features	10%

3. Objectives

- Build a genuinely hybrid scorer where both semantic and keyword signals are independently computed and meaningfully combined.
- Return every candidate ranked, not a filtered top-N, with a transparent final score.
- Generate clear, specific, non-generic explanations for top-ranked candidates.
- Keep the entire system — parsing, matching, scoring, and explanation — running locally with zero external API or network dependency.
- Handle real-world messiness gracefully: typos, inconsistent formatting, varied date styles, and candidates who submit more than one resume file.
- Implement both bonus features to a genuinely useful standard, not as a token gesture.
- Present results through a clean, professional recruiter-facing interface rather than a raw data dump.

4. System Architecture

4.1 High-Level Pipeline

Nexora is organized as a linear pipeline with two independent scoring branches that converge into a single hybrid score, followed by two independent reasoning layers (candidate-level explanation and JD-level bias review) that both feed the same Streamlit interface. Every stage runs in-process, on the machine running the app — there is no network call anywhere in this flow.

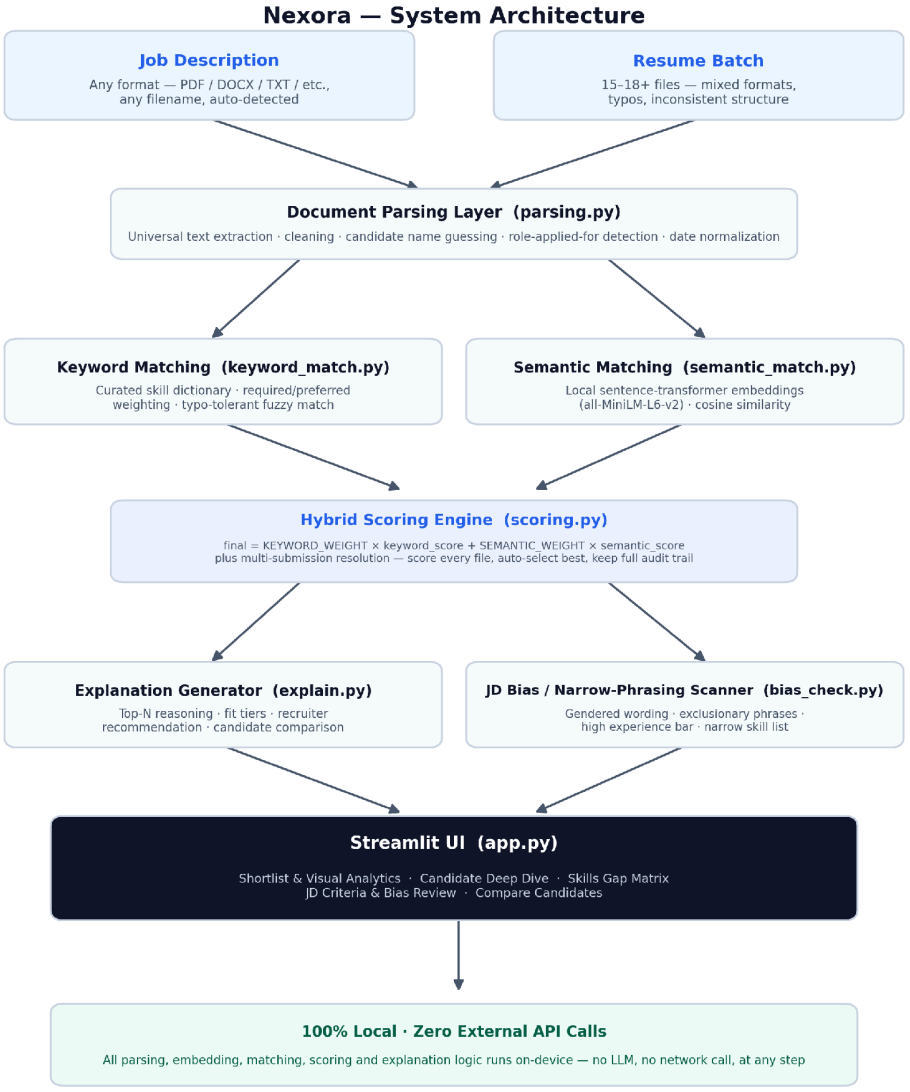


Figure 1: Nexora end-to-end system architecture, from raw JD/resume input to the Streamlit UI.

4.2 Module Breakdown

File	Responsibility
app.py	Streamlit UI — dashboard, candidate deep dive, skills matrix, JD review, candidate comparison, and all presentation-layer styling.
config.py	Central configuration: scoring weights, embedding model name, JD/resume data paths (auto-detected), explanation limits.
parsing.py	Universal document parsing across 14 file formats, text cleaning, candidate name extraction, role-applied-for detection, date normalization.
keyword_match.py	Curated skill dictionary, required/preferred section detection, typo-tolerant fuzzy matching.
semantic_match.py	Local sentence-transformer embedding generation, text chunking, cosine similarity scoring.
scoring.py	Combines keyword and semantic scores into the final hybrid score; resolves multi-submission candidates; produces the final ranked list.
explain.py	Template-based candidate explanations, fit-tier and recommendation logic, candidate-vs-candidate comparison.
bias_check.py	Heuristic JD bias and narrow-phrasing scanner.

5. Methodology

5.1 Keyword Matching

A curated dictionary maps roughly 60 canonical skills (languages, frameworks, databases, cloud/DevOps tools, testing practices, and core CS concepts) to their common surface forms — for example, “React” matches “react”, “react.js”, and “reactjs”. The job description is split into sections, and each section is classified as required, preferred, or unlabeled by matching its header against a list of common phrasings (“requirements”, “must have”, “nice to have”, “preferred qualifications”, and similar). Skills found under a required-sounding header receive twice the weight of those found elsewhere, so a candidate's keyword score reflects not just which skills they have, but which of the JD's actual priorities they satisfy.

Alternative-skill groups (for example, any one of MongoDB / SQL / PostgreSQL / MySQL / SQLite / Firebase / DynamoDB counting as satisfying a general database requirement) prevent the system from unfairly penalizing a candidate who used a different but equivalent tool than the one literally named.

5.2 Semantic Matching

The JD and every resume are embedded using a locally-run sentence-transformer model (all-MiniLM-L6-v2), with long documents chunked and averaged so meaning is captured across the full text rather than only the first few hundred words. Similarity between the JD embedding and a resume's embedding is computed with cosine similarity. This is what lets the system recognize that “built REST APIs with Express and MongoDB” is relevant to a Node.js backend role even when the JD never uses the word “Express” — the model captures conceptual relevance, not just literal word overlap. All embedding computation happens on-device; no text is sent anywhere.

5.3 Hybrid Scoring Formula

final_score = KEYWORD_WEIGHT × keyword_score + SEMANTIC_WEIGHT × semantic_score

Both component scores are pre-normalized to the 0–1 range before combining, so the weights in config.py directly and transparently control how much each signal contributes. The default is an even 50/50 split, and the weight is also exposed as a live slider in the UI — a recruiter can shift emphasis toward stricter keyword matching or broader contextual fit and see the ranking recompute immediately, without touching code.

Fit-Tier Categorization

Tier	Final Score	Required-Skill Coverage
Strong Match	≥ 0.72	≥ 70%
Moderate Match	≥ 0.50	≥ 40%
Partial Fit	≥ 0.30	—
Low Fit	below 0.30	—

5.4 Candidate Name & Role-Applied-For Extraction

Candidate names are guessed from the top of each resume using a shape-based heuristic (1–4 capitalized tokens, no digits, not a known job-title/section-header stopword), scanning past contact-info and header lines rather than naively taking the first line. Placeholder/synthetic resumes that carry an explicit “Candidate N” tag instead of a real name are detected and treated as their own identifier immediately, so they never collide with each other or with an unrelated section heading further down the page.

The role a resume targets is resolved in priority order: an explicit “Candidate N – Role” tag when present; otherwise a dictionary of ~30 common role phrases matched against the filename (filenames are very often deliberately named after the role, e.g. “web_dev_aditya_kulkarni.pdf”); otherwise the same dictionary matched against the resume's own header and summary text; and finally a cleaned-up version of the filename itself as a last resort. This resolved cleanly for 100% of the internal test batch with no blank or unusable labels.

5.5 Multi-Submission Handling

A candidate who sends in more than one resume file (for example, a PDF and a separately formatted text copy) is never silently deduplicated with a “(2)” suffix, and never has one submission arbitrarily discarded. Every submission is parsed and scored independently; the highest-scoring submission represents that candidate in the main ranking, while the full comparison — every file they sent, every score, and which one was auto-selected and why — is kept and surfaced in its own UI section. Only byte-identical resume text under the same guessed name is treated as a true duplicate and skipped.

5.6 Robustness to Messy or Inconsistent Formatting

- Typo tolerance: a conservative fuzzy match combines a similarity-ratio check with a dedicated adjacent-character-transposition detector (catching real typos like “pyhton” → Python or “dcoker” → Docker) — validated against roughly 70 genuine typo pairs and 70 commonly-confusable English word pairs with zero false positives.
- Date normalization: numeric date formats (03/2021, 2021-03, 2021/03, …) are collapsed to a single consistent “Mon YYYY” spelling before scoring.
- Bullet-glyph and hyphenation cleanup: varied bullet characters are normalized, and words split across a line-break hyphen are rejoined, so downstream matching sees clean text.
- Universal parsing across 14 file formats (PDF, DOCX, DOCM, DOTX, TXT, XML, JSON, HTML, RTF, Markdown, CSV, TSV), so format inconsistency across a resume batch is a non-issue.

6. Explainability Layer

For the top-ranked candidates (configurable beyond the required top 3 via TOP_N_EXPLANATIONS in config.py), Nexora generates a structured explanation directly from the computed scores and skill sets — no generative text model is involved anywhere in this layer. Each explanation states which signal (keyword or semantic) primarily drove the ranking, lists matched skills, lists missing required skills, and assigns a recruiter recommendation:

Recommendation	Logic
Highly Recommended	Strong Match tier, or a final score ≥ 0.70 with no missing required skills.
Consider	At most 2 missing required skills and a final score ≥ 0.50 — strong background, worth a quick verification.
Potential Backup	Final score ≥ 0.40 — meets baseline criteria but has notable gaps versus top candidates.
Not Recommended	Final score below 0.40 — substantial misalignment with core requirements.

7. Bonus Feature — JD Bias & Narrow-Phrasing Review

This heuristic, fully offline scanner rereads the job description itself and flags language worth a second look — framed explicitly as a prompt for human judgment, never as a verdict. It checks for masculine- and feminine-coded word usage (word lists adapted from Gaucher, Friesen & Kay, 2011, “Evidence That Gendered Wording in Job Advertisements Exists and Sustains Gender Inequality”, Journal of Personality and Social Psychology), exclusionary phrases with a plain-language rationale and suggested fix for each (e.g. “native English speaker”, “recent graduate”, “young and energetic”, “digital native”, “culture fit”, informal bravado terms like “rockstar”/“ninja”/“guru”, and gendered pronouns used without a neutral alternative), an unusually high flat years-of-experience bar (8+ years), and an overly long required-skills list (10 or more) that could narrow the applicant funnel unnecessarily.

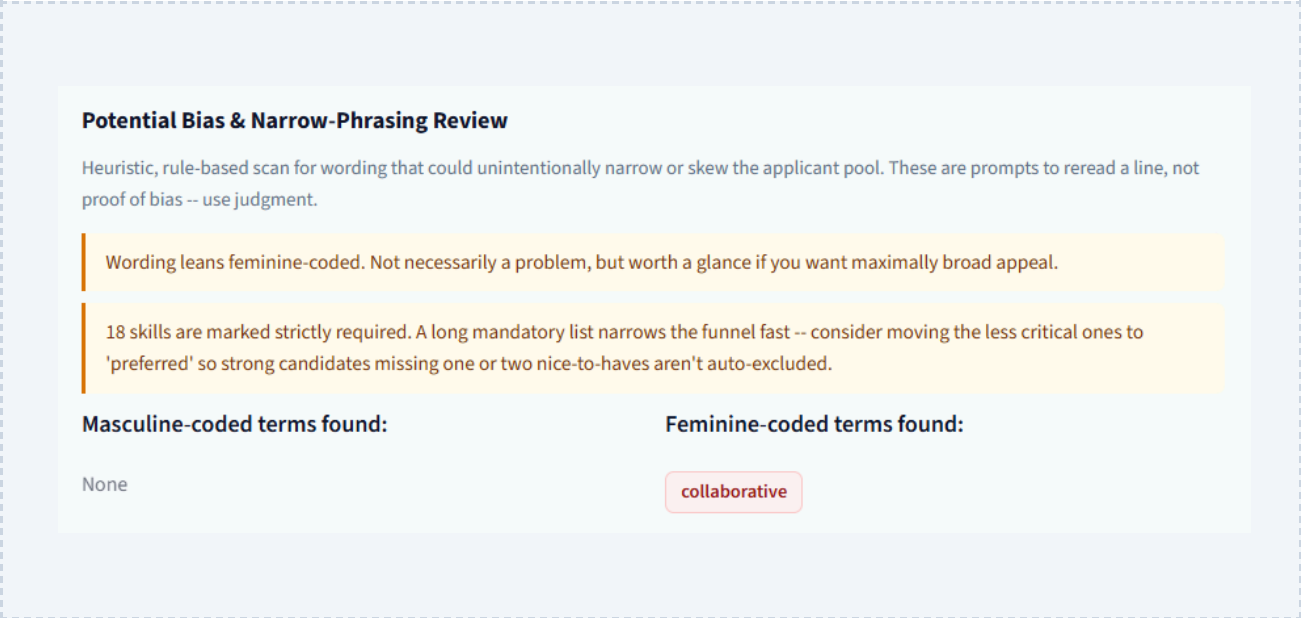


Figure 2: Job Description Criteria tab — Bias & Narrow-Phrasing Review

8. Bonus Feature — Recruiter "Compare Candidates" Tool

This answers the exact recruiter question posed in the problem statement — "Why is Candidate X ranked above Candidate Y?" — with a natural-language and tabular breakdown built entirely from already-computed scores, not a generative model. A recruiter picks any two candidates; the tool states the score gap and which candidate leads, identifies whether the gap is driven mainly by keyword or semantic signal, lays out a side-by-side metrics table (final, keyword, semantic, required and preferred coverage, fit tier) with the higher value highlighted per row, and lists skills exclusive to each candidate alongside any required skills each one is still missing.

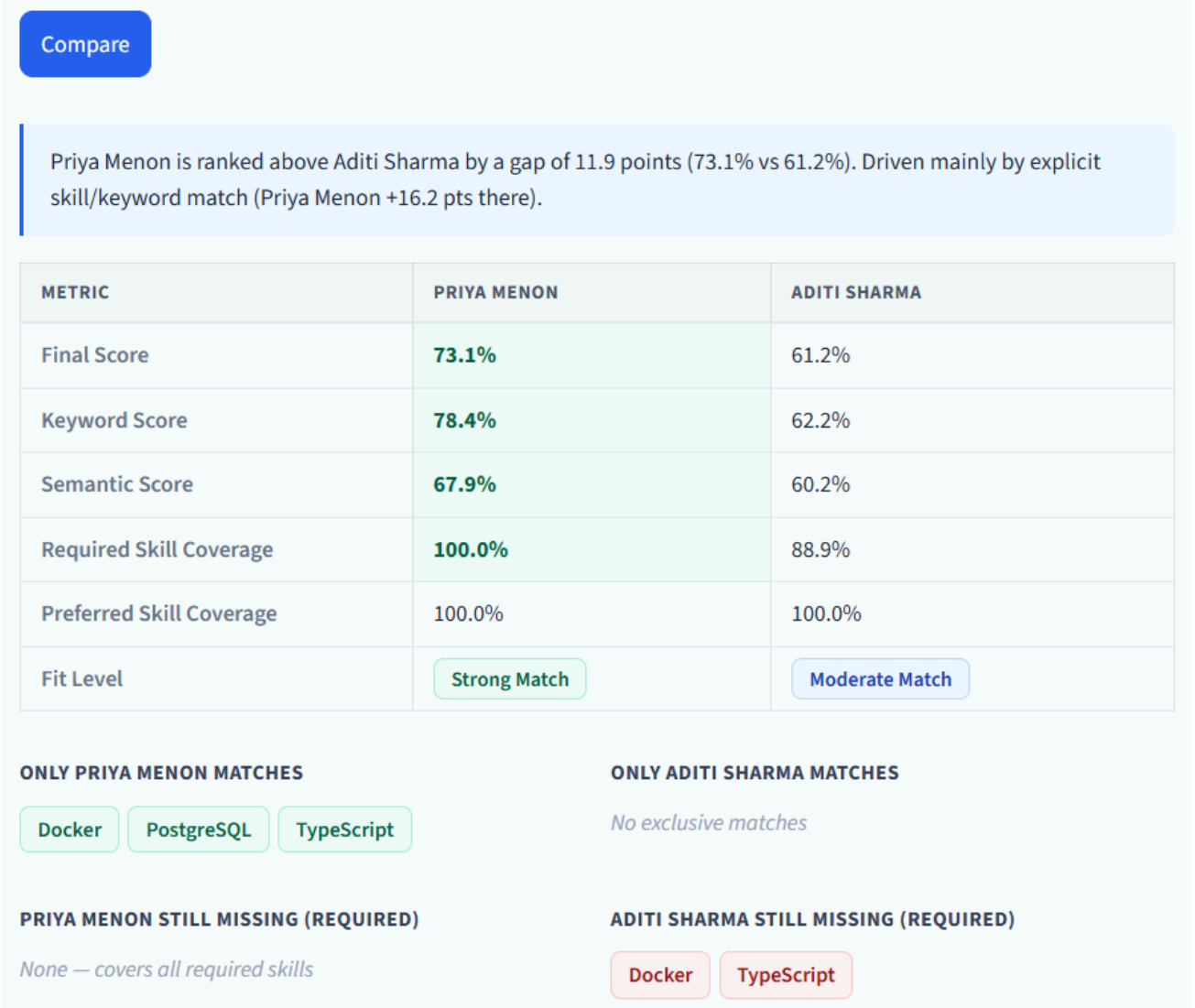


Figure 3: Compare Candidates tab — completed comparison

9. User Interface Walkthrough

Nexora presents results through five tabs, designed to read as a recruiter-facing product rather than a raw data science notebook. Two of the five (Job Description Criteria and Compare Candidates) are captured in the bonus-feature sections above; the remaining three are captured here.

9.1 Shortlist & Visual Analytics

The primary tab: summary KPI cards, ranking charts (capped and stacked vertically for readability rather than squeezed side-by-side), and the full candidates ranking table — including the Role Applied For column and row highlighting (red for placeholder/nameless entries, orange for zero matched skills) for fast recruiter triage.

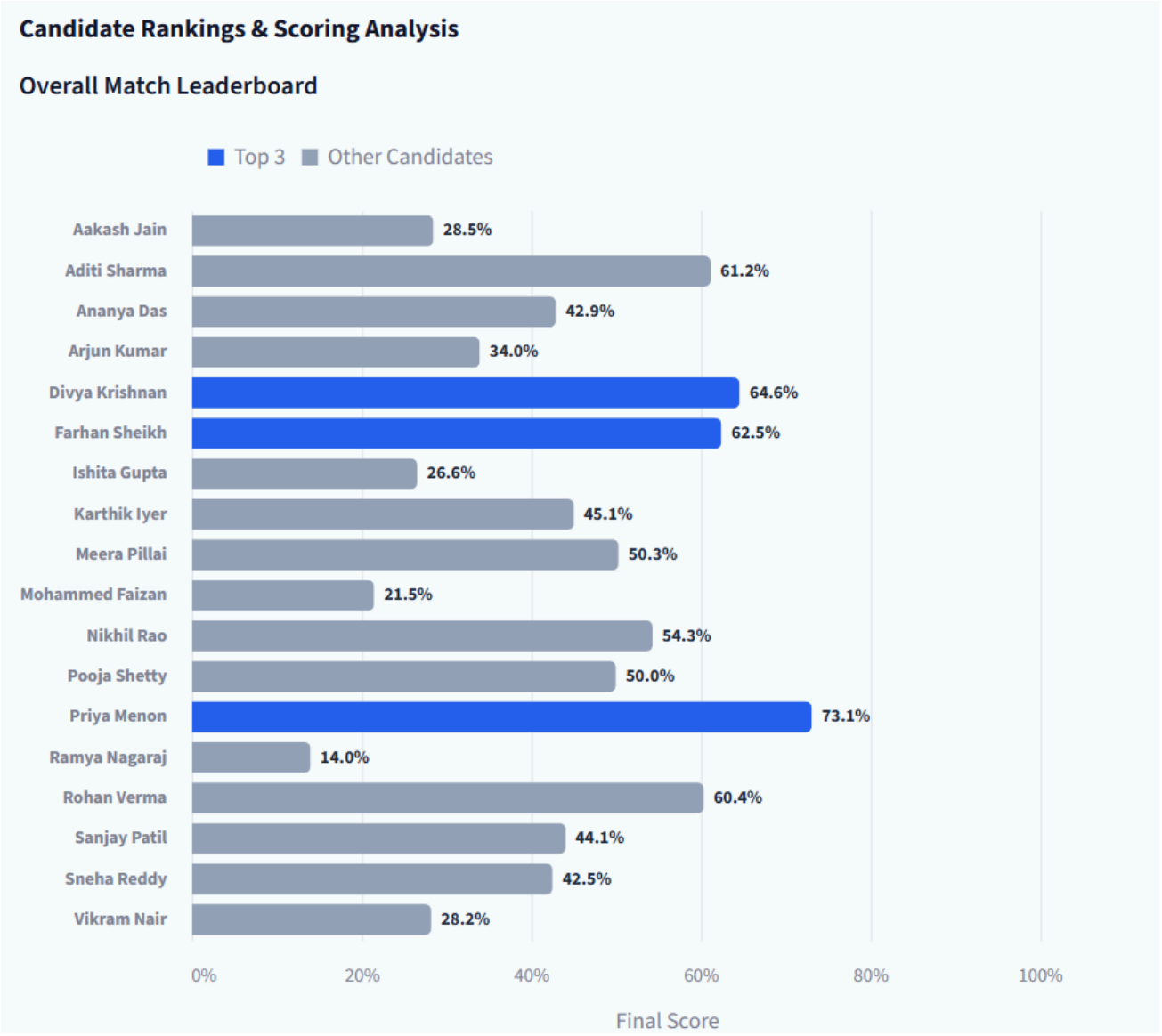
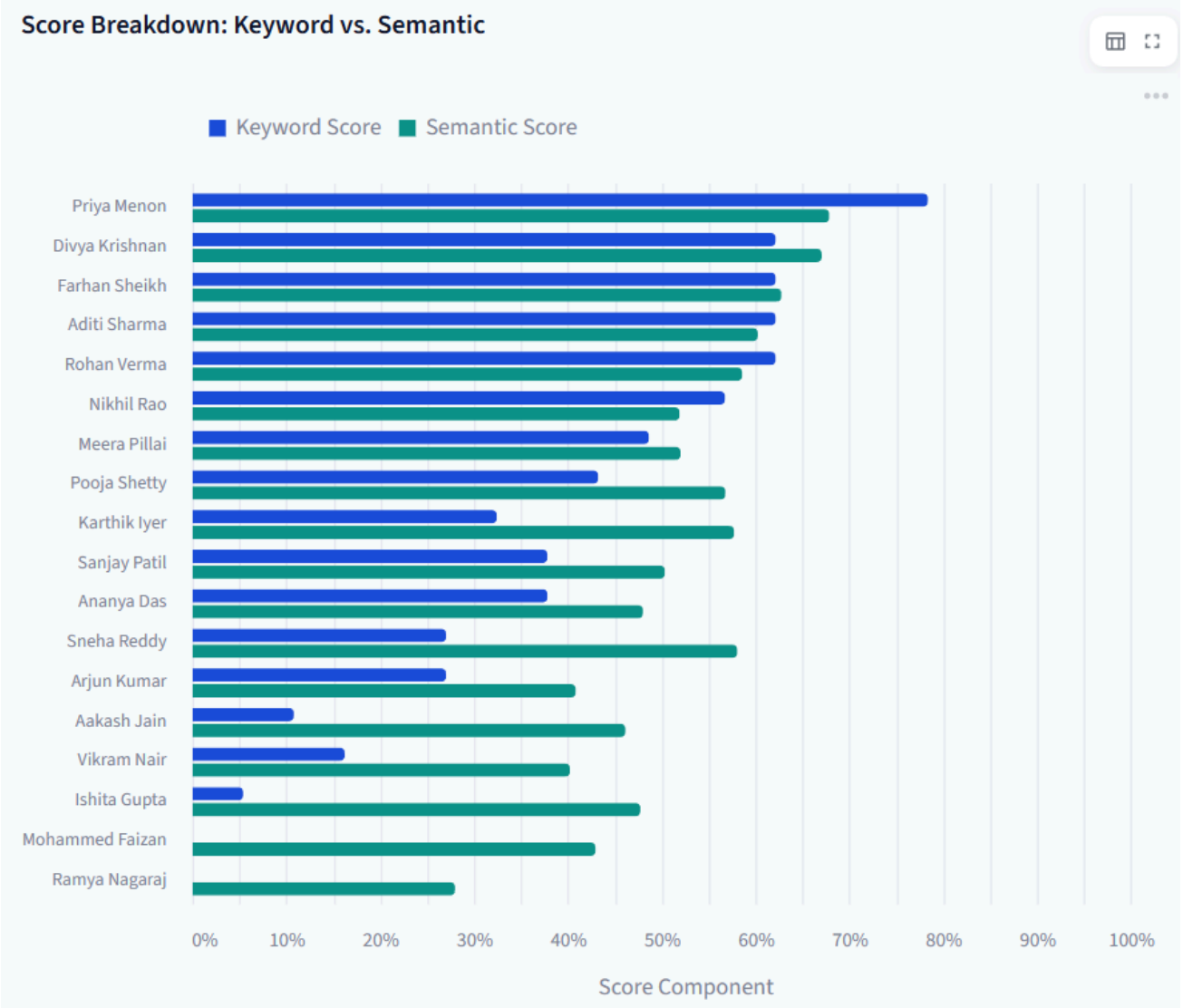


Figure 4: Shortlist & Visual Analytics tab — full view



9.2 Multiple-Submission Transparency

When one or more candidates submitted more than one resume file, this section (below the main ranking table) lists exactly which files each of them sent, how each one scored, and which was auto-selected: so the best-submission selection is never a black box.

9.3 Candidate Deep Dive & Feedback

Rich, per-candidate cards for the top-ranked candidates: overall/keyword/semantic match metrics, the fit-tier badge, the driving signal, matched and missing skills, and the recruiter recommendation; essentially the required top-3 explanation, presented as a readable profile rather than a raw score.

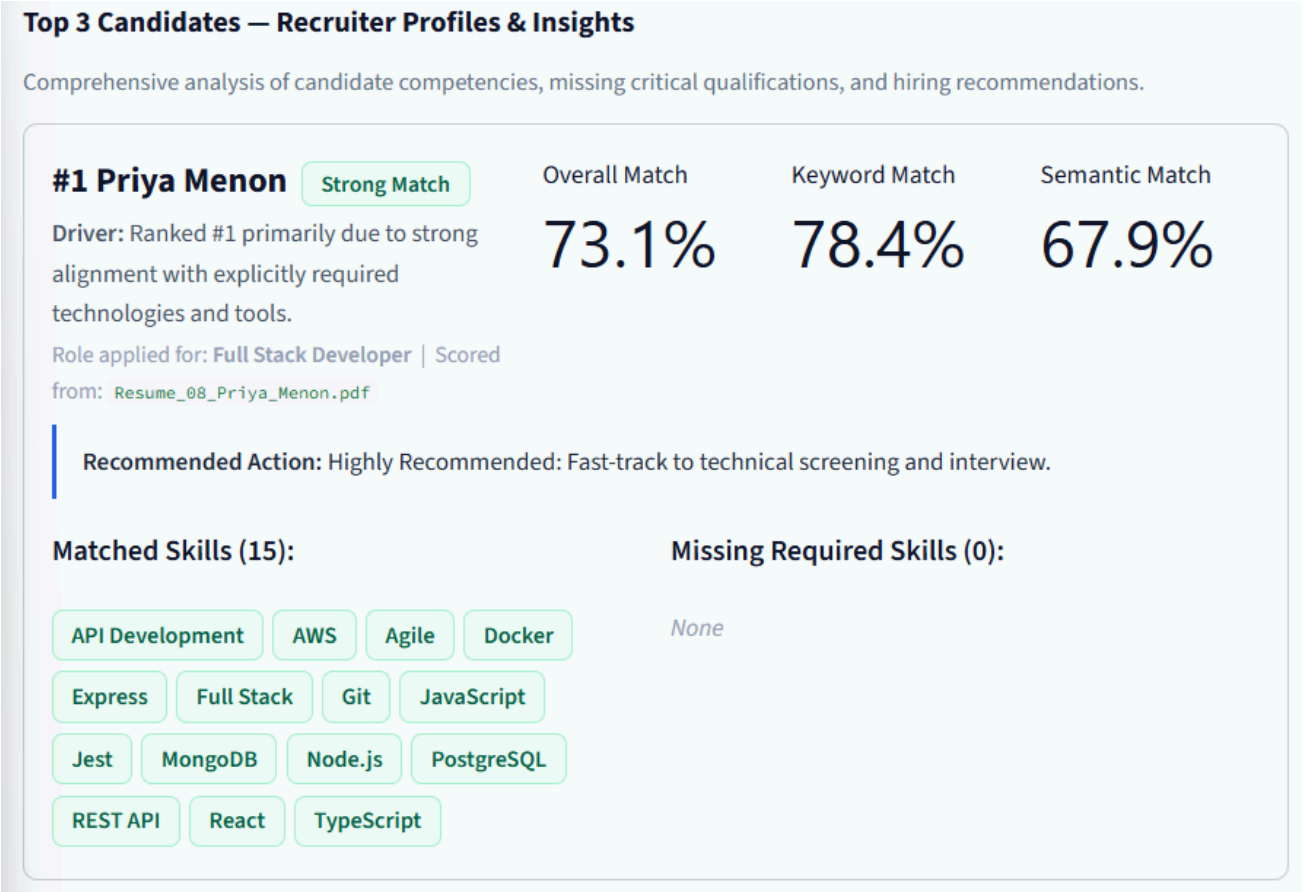
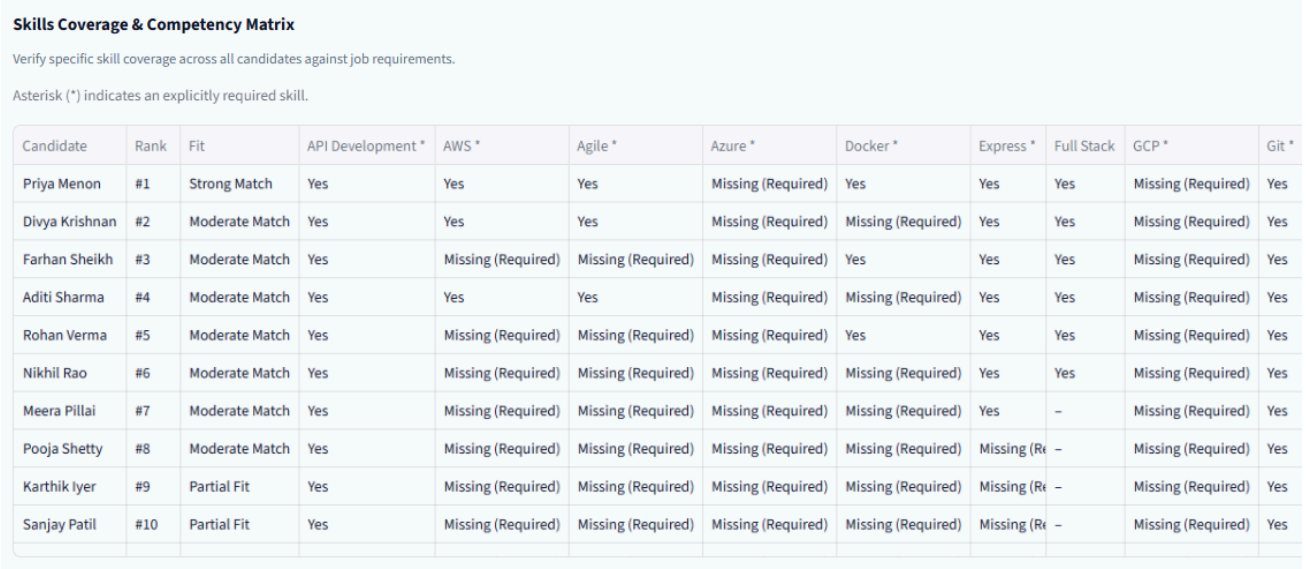


Figure 5: Candidate Deep Dive & Feedback tab — one expanded card

9.4 Skills Gap Matrix

A candidate-by-skill matrix (Yes / Missing / –) showing, at a glance, exactly which JD-relevant skills each candidate does and doesn't have — useful for a recruiter comparing several similarly-scored candidates side by side.



Skills Coverage & Competency Matrix

Verify specific skill coverage across all candidates against job requirements.

Asterisk (*) indicates an explicitly required skill.

Candidate	Rank	Fit	API Development *	AWS *	Agile *	Azure *	Docker *	Express *	Full Stack	GCP *	Git *
Priya Menon	#1	Strong Match	Yes	Yes	Yes	Missing (Required)	Yes	Yes	Yes	Missing (Required)	Yes
Divya Krishnan	#2	Moderate Match	Yes	Yes	Yes	Missing (Required)	Missing (Required)	Yes	Yes	Missing (Required)	Yes
Farhan Sheikh	#3	Moderate Match	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Yes	Yes	Yes	Missing (Required)	Yes
Aditi Sharma	#4	Moderate Match	Yes	Yes	Yes	Missing (Required)	Missing (Required)	Yes	Yes	Missing (Required)	Yes
Rohan Verma	#5	Moderate Match	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Yes	Yes	Yes	Missing (Required)	Yes
Nikhil Rao	#6	Moderate Match	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Required)	Yes	Yes	Missing (Required)	Yes
Meera Pillai	#7	Moderate Match	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Required)	Yes	–	Missing (Required)	Yes
Pooja Shetty	#8	Moderate Match	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Re	–	Missing (Required)	Yes
Karthik Iyer	#9	Partial Fit	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Re	–	Missing (Required)	Yes
Sanjay Patil	#10	Partial Fit	Yes	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Required)	Missing (Re	–	Missing (Required)	Yes

Figure 6: Skills Gap Matrix tab

10. Compliance with Hackathon Requirements

Requirement	Status	How Nexora Addresses It
Semantic + keyword matching (35%)	Done	Independent local semantic (embedding cosine similarity) and keyword (dictionary/regex) scores, genuinely combined via a config-adjustable weighted formula — not a single black-box call.
Quality/sensibility of ranking (20%)	Done	Full ranked list for every candidate, fit-tier categorization, required-vs-preferred weighting, alternative-skill-group collapsing.
Top-3 explanations (20%)	Done	Structured, template-generated reasoning: driving signal, matched/missing skills, recruiter recommendation — extendable beyond top 3.
Working end-to-end demo (15%)	Done	Single-command Streamlit app, auto-detects JD/resumes from the data folders regardless of filename, cached pipeline for fast reruns.
No LLM-API scoring (core constraint)	Done	Zero external API calls anywhere — semantic scoring uses a locally-run open-weight model, keyword scoring is pure regex/dictionary logic, explanations are template-based.
Bonus: JD bias / narrow-phrasing flagging	Done	Heuristic scanner covering gendered wording, exclusionary phrases, high experience bars, and narrow required-skill lists.
Bonus: recruiter Q&A / chat layer	Done	“Compare Candidates” tool answers “Why is X ranked above Y?” in natural language, with a supporting metrics table.
Bonus: messy/inconsistent formatting	Done	Typo-tolerant matching, date normalization, bullet/hyphenation cleanup, 14-format universal parser, multi-submission transparency.

11. Offline Execution & Privacy

Every stage of the pipeline — document parsing, keyword matching, semantic embedding, scoring, explanation generation, and bias scanning — runs as local Python code on the machine running the app. The only model involved is a small, open-weight sentence-transformer (all-MiniLM-L6-v2) loaded and run entirely on-device; no resume text, JD text, or derived data is ever transmitted to an external API, LLM, or third-party service. This was treated as a hard constraint throughout development, not an afterthought.

12. Testing & Validation

- End-to-end pipeline validated against the provided sample batch and an expanded internal test set — including deliberately placeholder/edge-case resumes — to confirm ranking behaves correctly under both clean and messy conditions.
- Typo-tolerant fuzzy matching validated against roughly 70 genuine typo pairs and 70 commonly-confusable English word pairs, with zero false positives after combining a similarity-ratio threshold with a dedicated adjacent-transposition check.
- Candidate name and role-applied-for extraction verified across the full internal sample batch, with 100% of resumes resolving to a usable, non-blank label.
- Multi-submission grouping and best-submission selection verified against synthetic multi-file candidates to confirm no submission is ever silently dropped or merged.
- JD bias scanner verified against known trigger phrases, including phrases deliberately split across a PDF line-wrap, to confirm whitespace normalization catches them correctly.
- UI verified end-to-end in a live browser session across all five tabs after each round of changes.

13. Challenges Faced & Solutions

Challenge	Root Cause	Solution
Dark-theme text invisibility	Custom CSS assumed a light background that the deployed theme didn't guarantee.	Pinned the app to a light theme explicitly via .streamlit/config.toml rather than patching individual colors.
Congested, hard-to-read charts	All candidates plotted in half-width columns regardless of batch size.	Capped chart display count and switched to vertically stacked, full-width charts.
Duplicate candidates from multiple submissions	Original logic appended a “(2)” suffix or risked discarding a submission outright.	Redesigned scoring to evaluate every submission independently, auto-select the best, and expose the full comparison in the UI.
Placeholder resumes silently merged into one identity	Nameless “Candidate N” resumes all fell through to the same wrong guessed name, collapsing distinct applicants into one leaderboard row.	Added explicit detection of the “Candidate N” tag as an authoritative identifier, checked before the general name-guessing heuristic.
Typo tolerance without false positives	A similarity-ratio threshold loose enough to catch real typos also matched unrelated real words (e.g. “shift”/“swift”).	Combined the ratio check with a separate, precise adjacent-character-transposition detector, tuned against ~140 real test pairs.

14. Future Enhancements

- Exportable PDF/CSV shortlist reports directly from the UI, for sharing outside the app.
- A user-editable skill dictionary and role dictionary via the UI, instead of code-level configuration.
- Support for ranking against multiple job descriptions in a single session, with per-JD leaderboards.
- Optional resume-side feedback generation — concrete suggestions a candidate could act on to close their skill gaps.
- Configurable, swappable local embedding models for teams wanting a different accuracy/speed trade-off.

15. Conclusion

Nexora meets every core requirement of the InternLoom AI Hackathon problem statement — genuine hybrid semantic and keyword matching, a full explainable ranked shortlist, and clear top-candidate reasoning — while implementing both optional bonus features to a functionally complete standard. Equal weight was placed on the constraint that mattered most throughout development: a system that is transparent and fully local, with no shortcut through an external LLM API at any stage of the actual matching and scoring logic.

Appendix A — Tech Stack & Dependencies

Component	Purpose
Streamlit	Web UI framework — dashboard, tabs, interactive controls.
sentence-transformers (all-MiniLM-L6-v2)	Local embedding model for semantic similarity scoring.
scikit-learn	Cosine similarity computation.
pandas	Tabular data handling for the ranking table and matrices.
Altair	Chart rendering for the visual analytics tab.
pdfplumber	PDF text extraction.
NumPy	Numerical operations supporting embeddings and scoring.

Appendix B — Full File Structure

Path	Contents
app.py	Streamlit UI and presentation layer
config.py	Scoring weights, model name, data-path auto-detection
parsing.py	Document parsing, name/role extraction, text cleaning
keyword_match.py	Skill dictionary and typo-tolerant keyword matching
semantic_match.py	Local embedding generation and cosine similarity
scoring.py	Hybrid scoring and multi-submission resolution
explain.py	Candidate explanations and candidate comparison
bias_check.py	JD bias / narrow-phrasing heuristic scanner
requirements.txt	Python dependencies
.streamlit/config.toml	UI theme configuration
data/jd/	Job description input (any supported format/filename)
data/resumes/	Resume batch input (any supported format)